

The Brutal Q&A

Every hard question about Inayat — in points you can actually remember

No passages to re-read. Short points, in order. Say the bullets in your own words — don't recite them.

0. The name — say this before anything else

Q: What does “Inayat” mean, and why did you choose it?

- Inayat means “care” in Urdu.
- We named it that because the whole prototype exists to give every patient the caring attention a rushed two-minute doctor visit can't.

Q: Which dataset did you train it on?

- None — we didn't train a model at all.
- The danger-check and follow-up questions are fixed rules, not learned from data.
- The “AI” part is a real, already-trained model (Claude) doing live reasoning — not something we trained ourselves.
- Our only “data” is a small, 14-sentence reference list we wrote ourselves, and openly say isn't officially certified yet.

Q: What languages does it actually support?

- English, Hindi, and Telugu — both on-screen text and voice.
- Picked Telugu specifically since this problem statement's example district (Kurnool) is Telugu-speaking.

Q: Does it work with no internet at all?

- The page itself still needs one internet load, like any website.
- But once loaded: if the AI can't be reached, it doesn't crash or freeze.
- It safely falls back to “route this to a human,” instead of guessing.

Q: Can a patient ask you to delete their data?

- Not yet, honestly — there's no delete button today.
- A doctor can correct or amend a case, but records aren't destroyed.
- A real, known gap, not something we're hiding.

Q: Is the code actually public? Can we check it ourselves?

- Yes — real GitHub repo, real commit history, nothing hidden.
 - github.com/azlanabyssal-cloud/carepilot
-

1. The problem — from zero, no background needed

Picture this first, before any numbers: someone is sick, waits hours in a government hospital line, and gets a doctor who has only minutes before the next patient. That's the whole problem, in one picture. The numbers below just prove it's real and everywhere.

Q: What is the actual problem?

- A government-hospital doctor sees a patient for about **2 minutes**.
- Not enough time to ask, listen, AND read old prescriptions.
- The “listening” part is what gets rushed first.
- But real studies say: careful listening is the #1 reason doctors get the diagnosis right.

Q: Why does this matter for real people, not just in theory?

- 2 out of every 3 Indians live in villages.
- Only about 1 in 4 doctors work in villages.
- Least access to a doctor = also the least time once you get one.

Q: How did you solve it, in one sentence?

- We talk to the patient **before** the doctor does.
 - Type, speak in their own language, or photo an old prescription.
 - We turn that into one clean, organized note.
 - By the time the doctor walks in, the listening is already done.
-

2. The pipeline — 7 steps, and why each one exists

- **1. Consent.** Patient must agree first — system refuses to save without it.
- **2. Patient talks.** Type, speak, or photo — not everyone types well or speaks English.

- **3. Smart follow-up questions.** Cough → phlegm/colour questions. Rash → spreading/new-soap questions. Pain → the standard 8-question method doctors already use. One list for everything would be lazy.
 - **4. Danger check FIRST.** Checks exact words against a dangerous-symptom list. Zero AI needed. Zero internet needed. A dangerous case should never depend on AI having a good day.
 - **5. AI helps (if not obviously dangerous).** Suggests urgency. No AI available? A safe backup kicks in instead of the app failing.
 - **6. Second, independent double-check.** Compares words against real medical guidelines. Can only push urgency UP, never down. A wrong “less serious” correction is far more dangerous than extra caution.
 - **7. Doctor decides.** Everything saved. Doctor reads it, can change anything. AI never diagnoses. AI never prescribes.
-

3. The engineering questions

Q: How much space (GB), and where does it run?

- Real app code: about **4.5 MB** — smaller than one phone photo.
- Libraries it actually needs to run: about **290 MB**, measured directly.
- One heavy library (PyTorch, for a feature not wired into the live app yet) left OUT on purpose — saves ~5 GB of dead weight.
- Hosted on **Render**, a cloud host, free tier.
- Free tier sleeps after 15 min idle, takes 30–60 sec to wake up — we say this openly.

Q: What parameters or choices did you make, and why?

- Guideline matching: simple word-matching, not a giant AI model.
- Why: our reference list is only a few dozen short passages — a big model gains nothing here.
- We trust only the **single best** guideline match, not the top 3.
- Why: a weak 3rd-place match once wrongly escalated “mild knee pain” to EMERGENCY.
- We caught it ourselves, tested it, fixed it.

Q: What tech did you actually build this with?

- Python for the server.
 - Plain HTML/JavaScript for the screen — no heavy framework, loads fast on cheap phones.
 - A small database to save cases.
 - Claude (AI model) for language understanding.
 - Bhashini — real Indian-language voice tool.
 - ABDM — India's real health-ID system.
-

3b. The raw “why this and not that” round — college-style questions

Q: Why Python/FastAPI and not Java (Spring Boot)?

- Java's fine — it's a real, valid choice too, just not ours.
- Python plugs straight into the AI/ML tools we needed: Anthropic's SDK, scikit-learn, Tesseract OCR — no cross-language glue code.
- FastAPI auto-validates every request's shape (via Pydantic) before our code even runs — several real bugs this project found were exactly bad/malformed input, caught right there.
- Less boilerplate, faster to build correctly in a hackathon timeline.

Q: Why SQLite and not MySQL or a “real” database?

- SQLite is a real database — just one file, zero separate server to set up.
- Sized to what this actually needs right now: one well-defined table, a demo/pilot scale.
- Honest, named next step: move to Postgres before any real multi-hospital rollout.

Q: Why plain HTML/JavaScript and not React or Angular?

- The user we're designing for is a patient on a cheap Android phone at a rural kiosk.
- No framework, no build step — the page just loads, instantly, with nothing to download first.
- Developer convenience lost out on purpose to the actual end user's phone and network.

Q: Is this actually a REST API? What does that even mean here?

- Yes, genuinely — every feature is a real HTTP endpoint (e.g. POST /assess, POST /case-intake).
 - Each one takes structured JSON in, returns structured JSON out — the standard REST shape.
 - FastAPI also auto-generates live API documentation from the same code — nothing hand-written or able to drift out of sync.
-

4. Why we're different — and the ONE thing nobody else has

Q: How is this better than another team's health chatbot?

- Most chatbots: one AI model + a disclaimer.
- Ours: a safety net that doesn't need the AI at all.
- Keeps working with zero internet.
- Saves the case for the doctor to reopen later.
- Connects to real government health systems.

Q: The ONE thing you have that basically nobody else has?

- A safety check that can only get MORE careful, never less.
- Works even if the AI is completely wrong, or fully offline.
- Most student projects trust the AI's answer directly.
- We built one that never fully trusts it.

Q: Isn't this just ChatGPT with extra steps?

- ChatGPT: no double-check, nothing saved, stops with no internet.
 - We built 3 separate real things ChatGPT doesn't have.
 - Not one clever prompt — real engineering.
-

5. Grabbing attention — the pitch, and the app itself

Q: How will you grab attention when you present this?

- Open with the real number, not a claim.
- “2 minutes per patient” — one of the lowest in the world.
- From a real study of 67 countries.
- Everyone's sat in that waiting room — it lands instantly.

Q: Once someone is looking at the app, what makes IT grab them?

- Homepage doesn't wait for a click.
 - A real example types itself out live, on its own.
 - Gets caught as an emergency — before the visitor touches anything.
 - Proof, not a promise.
 - Page reveals content smoothly while scrolling — feels alive, not static.
-

6. Limitations — proof we KNOW them, not guessed

A real limitation vs. a guess: we tested it and watched it fail, with our own eyes, before writing it down.

- **Old handwritten prescriptions read worse than clean typed ones.** Proof: same code, blurry image turned “13.0-17.0” into wrong “130-170.” We watched it happen.
 - **Our medical reference list isn't officially certified yet.** We wrote it ourselves as a real starting point — we say so.
 - **Voice and health-ID features never touched real government servers.** No test credentials exist for us — only public documentation.
 - **Offline voice recognition isn't very accurate yet.** We tested and measured it ourselves — every result gets flagged “needs a human check,” never silently trusted.
-

6b. Scalability, money, and rules — the questions we'd never thought to prep

Q: Will this actually scale past a demo, to a whole state or country?

- Frontend is one plain HTML/JS file — no framework, loads on a cheap Android phone.
- We don't fake government integration — we plug into real ones.
- ABDM (India's health-ID system) already has over **90 crore** real IDs on it.
- We're riding rails that are already built at national scale, not inventing our own.

Q: Isn't this just Practo or 1mg with a new name?

- Practo/1mg: book a doctor, order medicine, talk to a doctor remotely.
- Us: prepare what a patient says **before** they walk into an *existing* in-person visit.
- We don't replace the hospital visit — we make the few minutes inside it count more.
- Different job entirely, not a competing app.

Q: Who pays for this? What's the business model?

- Built as free public-health infrastructure, not a paid consumer app.
- Same shape as CoWIN or ABDM itself — adopted by a health system, not sold per-user.
- Honestly: we haven't signed a government partner yet — that's a real next step, not a claim.

Q: Is patient data handled legally? What about privacy law?

- Patient must actively give consent before anything is even saved — enforced by the system, not just a checkbox.
- Doctor's screen requires a real login — no login, no access.
- Full formal legal/compliance review hasn't happened yet — we say that honestly, not hide it.

Q: Could someone abuse this — fake symptoms to jump the queue?

- The system never grants anything by itself — no priority, no medicine, no appointment.
 - It only writes a note. A real doctor still decides everything that actually happens.
 - Faking words to an AI doesn't get you anything a doctor doesn't independently check first.
-

7. The extra brutal round

Q: What's the hardest real bug you personally found and fixed?

- Safety-check used to trust the “most serious” of the top 3 guideline matches.
- A weak, barely-related 3rd match once beat a correct, strong 1st match.
- Wrongly turned ordinary “mild knee pain” into EMERGENCY.
- We found it ourselves, proved it with a test.
- Fixed: only trust the single best match now.

Q: What if the AI gives a wrong answer during the demo?

- Danger-word check runs FIRST — zero AI needed for that.
- Second guideline check can only make things MORE careful.
- One wrong AI answer alone still can't create an unsafe result.
- Would need TWO independent systems wrong, in the same direction, at once.

Q: Couldn't a big company clone this in a week?

- The AI-chatbot part? Maybe, yes.
- The safety-net design, real integrations, real-world testing? No.
- That's not a weekend job — most teams skip it entirely.

Q: Is patient data actually safe?

- Doctor's screen sits behind a real login.
- Wrong passcode = the server itself refuses the data.
- Not just a hidden button someone could work around.

Q: Have you tested this on real patients in a real hospital?

- No — and we say that honestly.
- 367 automated tests, all currently passing.
- Real example cases we wrote ourselves.
- Real-patient testing is a genuine next step, not a claim we're making.

Q: If teammates give slightly different answers on stage, is that bad?

- No — only guessing looks bad.
 - Slightly different wording is fine.
 - Don't know something? Say so, offer to follow up.
-

8. The answers you say without thinking

The whole thing, in one line: “Inayat listens to a patient before the doctor does, and turns what they say into a safe, organized note — so the doctor's few minutes go to deciding, not repeating questions.”

Three reasons people should respect this project:

- We say what's NOT finished, out loud, before you catch us.
- The safety design doesn't trust one single thing to be right — it catches itself being wrong.
- Every real government feature we claim, we actually built and tested.

You've now read the real problem, the real pipeline, the real numbers, the real bugs we found ourselves, and the real limits. That's not luck. That's understanding. Go say it.

9. Final Stage Revision — every fact, zero fluff

Read top to bottom once, right before you walk up. Nothing here is new — it's page 1-4, compressed.

Name & pitch

- Inayat = “care” (Urdu). Named for giving real care a 2-min visit can't.
- Pitch: listens before the doctor does → clean note → doctor decides, doesn't repeat questions.

The problem (real numbers)

- ~2 min/patient in govt hospitals (67-country study).
- 2/3 of India rural, only ~1/4 of doctors rural.
- Careful listening = #1 driver of correct diagnosis.

Pipeline (7 steps)

- Consent → patient talks (type/voice/photo) → smart follow-ups → danger-word check (zero AI) → AI suggests level → guideline double-check (escalate-only) → doctor decides.

Numbers to have ready

- App code: 4.5 MB. Deploy deps: 290 MB. PyTorch (~5 GB) deliberately excluded.
- 367 automated tests passing. 11 authored eval cases. 3 languages (En/Hi/Te).
- Hosted on Render free tier — sleeps 15 min idle, 30–60s cold start.
- ABDM already has 90+ crore real IDs — we ride that scale, don't invent it.

Tech stack, one line each

- Python/FastAPI — auto request validation caught real bugs; no Java/Spring needed.
- SQLite — zero-setup, sized to today's scale; Postgres is the named next step.
- Plain HTML/JS — loads instantly on a cheap rural Android phone, no React/build step.
- Real REST API — POST /assess, /case-intake, etc., self-documented at /docs.
- Claude + Groq for reasoning, Bhashini for voice, ABDM for health-ID — all real, all wired in.

Why not X

- Not ChatGPT: we double-check, work offline, and save the case. ChatGPT does none of that.
- Not Practo/1mg: they book/deliver; we prep what's said before a visit that already exists.
- Not a trained model: zero dataset trained — deterministic rules + a real LLM doing live reasoning.

The ONE thing / the hardest bug

- The one thing: a safety net that can only get MORE careful, never less — and works with zero AI.
- Hardest bug: top-3 guideline match let a weak 3rd-place result beat a correct 1st — “mild knee pain” → wrongly EMERGENCY. Found it, tested it, fixed it (now: single best match only).

Limitations — say these unprompted

- Handwritten prescriptions read worse than typed (watched it happen, not guessed).
- Guideline list is self-written, not officially certified yet.
- Voice/ABDM never hit real government servers — docs only, no test credentials.
- No delete-my-data button yet. No signed government partner yet. No formal legal review yet.

Business, legal, abuse — the ones we almost forgot

- Free public-health infrastructure, like CoWIN/ABDM — not sold per-user.
- Consent required before saving anything; doctor login required to view anything.
- Can't be abused for priority — it only writes a note, a human decides everything real.

Closing lines

- One-liner: “Inayat listens before the doctor does, turns it into a safe note, so the doctor's minutes go to deciding, not repeating questions.”
- 3 respect reasons: says what's unfinished out loud; safety design catches its own mistakes; every government claim is actually built and tested.
- Don't know an answer? “Fair question — let me note it and follow up.” Never guess.